



Competitive Analysis of the Starbucks and Dunkin' Donuts Beverage Menus

A Step-by-Step Guide

A Signature Term Project designed for

BUS 303: Python for Business

Dr. Joseph Foy, CPA

CUNY at the School of Professional Studies, Online Business Programs

Revised 6/11/26

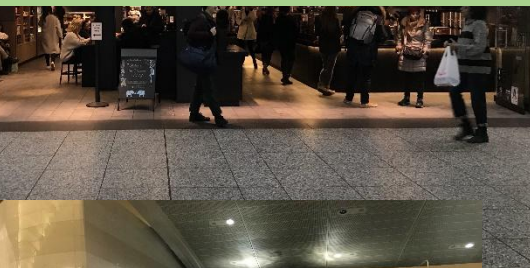


Table of Contents

Table of Contents	2
Introduction.....	5
Overview of the Term Project	6
Project Objectives:.....	6
Deliverable One: Cleaning and Preparing the Starbucks Menu for Analysis.	7
Deliverable One Tasks:	7
Task #1.1: Open a new Jupyter Notebook, rename it, download the Starbucks Beverage Menu.....	7
Task #2.1: Explore rows with all blank values and remove them.	8
Task #1.3: View the dataframe with the head function.....	8
Task #1.4: Remove the row with all blank entries.....	8
Task #1.5: Validate that the blank row(s) have been removed.....	8
Task #1.6: Explore the data types and correct any errors.....	8
Task #1.7: Look for a column with the incorrect data type, understand why it is incorrect, and change it to the correct data type.	8
Task #1.8: Change Suger_g type to float.	8
Task #1.9: Validate that the Sugar_g variable has been changed to a float.	9
Task #1.10: Explore the NaN values and remove the row with any NaN values.	9
Task #1.11: Assign a new dataframe df2 and remove the row with the NaN entry from the Saturated_Fat_g column in sbux1.....	9
Task #1.12: Validate that the NaN entry from the Saturated_Fat_g column has been removed.....	9
Task #1.13: Change a column name.....	9
Task #1.14: Validate that the Caffiene_mg column has been renamed corrected.....	9
Task #1.15: Write dataframe sbux3 to a new sbux4.CSV file.....	9
Task #1.16: Load the sbux.csv file back into the Jupyter Notebook and validate.	9
Deliverable Two: Cleaning and Preparing the Dunkin' Donuts Menu for Analysis.	9
Deliverable Two Tasks:	9
Task #2.1: Import the BUS_303_Dunkin_Donuts_Nutritional_Menu_Kaggle_2023.csv file.....	10
Task #2.2: Examine the Dataset Structure.....	10

[Click here to go back to Table of Contents](#)

Task #2.3: Explore Missing Values.	10
Task #2.4: Explore How Many Duplicate Records May Exist.	10
Task #2.5: Explore the Nature of Duplicate Records.	11
Task #2.6: Explore Unique Values.	11
Task #2.7: Examine the Beverage Categories.	11
Task #2.8: Explore Numerical Variables.	11
Task #2.9a-i: Identify Special Characters.	12
Task #2.10: Create a Cleaned Dataframe.	12
Task #2.11: Remove Food Items from the Category Rows.	12
Task #2.12: Verify duplicate item is removed.	13
Task #2.13a-b: Remove the special values inside each column.	13
Task #2.14: Validate removal of all special characters.	13
Task #2.15: Remove ' ' empty strings and replace with NaN values.	14
Task #2.16: Validate conversion of " empty strings with NaN values.	14
Task #2.17: Create a new dnkn3 object and drop the NaN values from the dataframe.	14
Task #2.18: Validate removal of the NaN values from the dataframe.	14
Task #2.19: Convert Data Types.	14
Task #2.20: Summary Validations	15
Task #2.21: Save the dnkn3 Python file to a CSV file.	15
Task #2.22: Reload the dnkn3 Python CSV file and save it as a new object, dnkn4.	16
Task #2.22: Summarize the issues with the Dunkin' Donuts dataset.	16
Deliverable Three: Comparing Starbucks and Dunkin' Donuts Menu for a Competitive Analysis	16
Deliverable Three Tasks:	16
Task #3.1: Create a horizontal bar chart comparing the Beverage Combination Portfolio Size of Starbucks and Dunkin' Donuts.	16
Task #3.2: Create a bar chart comparing average beverage calories.	17
Task #3.3: Create a bar chart comparing average sugar content.	19
Task #3.4: Create a bar chart comparing average protein content.	20
Task #3.5: Compare healthier beverage choices.	21
Task #3.6: Competitive Positioning Map.	23
Task #3.7: Zero-Calorie Beverages Count Analysis.	24
Task #3.8: Zero-Calorie Starbucks Beverage Analysis.	24

Task #3.9: Top 5 Most Caffeinated Starbucks Beverages.....	25
Task #3.10: Top 5 Most Sugary Dunkin' Donuts Beverages.....	26
Task #3.11: Choose your own visual.	27
Deliverable Four: Short Video Explaining Deliverables One through Three.	27
Task #4.1: Rehearse, record, and present a video presentation.	27
Task #4.2: Upload the video presentation.....	28
Deliverable Five: Reflection and Final Submission.	28
Task #5.1: Write a reflection in the Python file.	28
Task #5.2: Print an HTML copy of the Jupyter Notebook.	29
Task #5.3: Convert the HTML to a PDF file to upload it to ePortfolio.	29
Task #5.4: Email the GitHub links to yourself, your instructor and upload it into Blackboard.....	29
Task #5.5: Convert the HTML to a PDF file to upload it to ePortfolio.	29
Appendix A, Grading Rubrics.....	30
Deliverable One Grading Rubric	30
Deliverable Two Grading Rubric.....	32
Deliverable Three Grading Rubric	36
Deliverable Four Grading Rubric	38
Deliverable Five Grading Rubric	39

Introduction

This project is designed to address the lack of exposure dealing with real life businesses in at a high level active, or experiential learning environment. Many business classes do not offer opportunities to go outside of the classroom and try to incorporate what is learned in the books and students seemingly get stuck in the abstract, theoretical world. In this project, we attempt to provide an opportunity for students to make the connection between abstract, theoretical textbook learning to real, concrete experiences dealing with business analytical issues. This approach allows students to transform their learning from the abstract world to the real, concrete world by producing experiences that allow them absorb information and analyze a business more thoroughly.

This project addresses the theory-practice gap by exploring the Starbucks and Dunkin' Donuts Corporations, which are well-known, easily accessible publicly traded business entities. At the conclusion of the project, students will come to understand business analytics theory much better and how it is applied. This sets them up for the life-long journey to better understand information and improve chances to make more informed business decisions across various industries.

The instructions contained in this document are specifically written to allow students to connect the theoretical discussion to practical real world, concrete experiences driving toward a rich learning environment. It is hoped that students will take the opportunity to engage in this term project as a path to understanding abstract business analytics concepts, dealing with a real live business that has an impact in the communities where students may live/work.

It is hoped you will enjoy your experience.

If you see any errors or areas that need more clarity, please bring them to the attention of Dr. Joseph Foy at joseph.foy@cuny.edu. This document has undergone numerous edits based upon many suggestions from past students who have successfully completed earlier versions of the project. First time students are highly encouraged to give suggestions as well.

Good luck with this term project and let it enrich your understanding of Python!

Joseph Foy, CPA

Overview of the Term Project

An upstart competitor in the coffee business would like to better understand Starbucks' and Dunkin' Donuts beverage inventories. You have been hired as a consultant to help the upstart competitor to assist in this task. You could do the analysis in Excel, but the company hiring you would like you to share all the steps you take to analyze the data. In Excel, you could do this in Visual Basic coding. Basic Visual coding is the coding that is used behind the scenes in Excel. Here is the problem that you run against. Even if you possess the knowledge of how to code in Visual Basic, someone in the company that is hiring you would also have to know it, as well as anyone they would like to share it with. Visual Basic is not a widely used programming language. It is a Microsoft-specific product; therefore, not many programmers work with it. Fortunately, many programmers know how to code in open-access Python. Your client has obtained the extensive Starbucks' and Dunkin' Donuts beverage menus in a CSV file and provided them to you. Your overall objective is to import the file Starbucks' and Dunkin' Donuts beverage menus into Python, analyze and interpret the data, and make your code available to the company hiring you and present how you coded your analysis.

The project has six objectives, as follows:

Project Objectives:

- Project Objective 1: Organize complex data for business analysis.
- Project Objective 2: Explore business data for insights.
- Project Objective 3: Synthesize concepts to structure data to allow for business analysis.
- Project Objective 4: Share business data to different audiences.
- Project Objective 5: Present business data to make decisions.
- Project Objective 6: Reflect with a critical review of what students learned in the course and how Python can build your skill set.

To achieve the objectives, this project requires that you complete five deliverables. Each deliverable has specific tasks. Each deliverable is weighed based on the level of work involved. The term project is due at the end of the semester.

Grading rubrics are provided in [Appendix A, Grading Rubrics](#) of this document so students can see the weight of each deliverable and how they are to be scored. Students should become familiar with the grading rubrics as early as possible. If students have any questions, they are to email their instructor and explain the issue as succinctly as possible. The grade for the project will appear in the Course Learning Management System.

To be successful with this term project, students integrate the course materials with the project requirements. Therefore, students should become familiar with the course homework assignments, required course readings, supplemental readings, and course videos. If you are curious about Starbucks and its operations, here are helpful videos to understand Starbucks store level operations.

- [A Behind the Scenes Look at Starbucks Retail Operations](#)
- [Day in a Life: Nick Lister, Starbucks Barista](#)
- [Beverage Sequencing](#)

[Click here to go back to Table of Contents](#)

Deliverable One: Cleaning and Preparing the Starbucks Menu for Analysis.

Deliverable one focuses on term project objective to organize complex data for business analysis. This objective is designed to allow students to become familiar with organizing files within their own computer and load them into a commonly used Python IDE. This is normally the first step in the process to analyze business data.

Students will have to perform the tasks below. The first deliverable is divided into 17 tasks that must be performed in the exact sequence that is shown in this guide. It is also important that students become familiar with the grading rubric to understand how this deliverable is graded.

Deliverable One Tasks:

Task #1.1: Open a new Jupyter Notebook, rename it, download the Starbucks Beverage Menu

- A. Start Anaconda Navigator and launch a new Jupyter Notebook. It will open in a new browser tab. You will see a window that contains Files . In the upper right corner of this tab you will see a  button. Click on the New button and you will see a drop down box. Select  Python 3 (ipykernel). A new Jupyter Notebook file will open. In the upper left corner you will see  jupyter Untitled. Click the word Untitled. A pop up box titled  will appear. Rename the Jupyter Notebook from "Untitled" to the following name convention: LastName_FirstName_BUS_303_Signature_Term_Project. You will see that the file name Untitled has been replaced. If you go into your C:\Users\YOUR COMPUTER NAME FOLDER, you will see the LastName_FirstName_BUS_303_Signature_Term_Project.ipynb file. For MAC users, this should be /Users/ YOUR COMPUTER NAME FOLDER. This will now become the default location for your Jupyter Notebook file. The .ipynb extension means that it was generated from a Jupyter Notebook. Other IDEs assigned different file extension names.
- B. Next, move your cursor inside the Task 1.1 rectangular box and type “Import pandas as pd” to work with a package that can process dataframes.
- C. Import the BUS303_Signature_Term_Project_Starbucks_Beverage_Menu CSV file from either the Projects/Assignments folder in Brightspace or the GitHub account provided. Assign “sbux” (in lower case letters) as the file's object name. SBUX is the stock trading ticker symbol for Starbucks.

The BUS303_Signature_Term_Project_Starbucks_Beverage_Menu CSV file can be found in Brightspace, in the Projects/Assignments folder, or in the GitHub folder provided by your instructor. This is a “dirty” CSV file in the sense that it was originally downloaded as a PDF from a website and converted to CSV. However, when downloaded from a PDF file, it did not retain the column names correctly, contained entries that did not retain the PDF's

[Click here to go back to Table of Contents](#)

formatting, contained blank rows, and had numerous numeric values treated as objects (that is, strings). Your instructor cleaned up the CSV file quite a bit. However, it still contains items that need further cleaning to conduct a proper exploratory data analysis. Your job is to load this nearly clean file and make further adjustments. Once the file is loaded into Python, correct the issues so it is presentable as a “clean” CSV file and ready to share with anyone.

When the run button ► is clicked, a number will appear inside the brackets [] on the left side of the code block, indicating that the file has loaded.

D. Save the CSV file in your C:\Users\YOUR COMPUTER NAME FOLDER.

Task #2.1: Explore rows with all blank values and remove them.

A. Using the isnull and sum functions, check the total NaN entries for all columns in the dataframe.

Task #1.3: View the dataframe with the head function.

A. Using the head function, view the dataframe.

Task #1.4: Remove the row with all blank entries.

A. The how='all' will drop all rows with only NaN values.

Task #1.5: Validate that the blank row(s) have been removed.

A. Use the isnull and sum functions to validate that sbux2 has only one column remains with an NaN.

Task #1.6: Explore the data types and correct any errors.

A. Using the dtypes function, check the data types.

Task #1.7: Look for a column with the incorrect data type, understand why it is incorrect, and change it to the correct data type.

A. Using the iloc function, look for an incorrect data type.

Task #1.8: Change Suger_g type to float.

A. Hint: Use the astype(float) function.

[Click here to go back to Table of Contents](#)

Task #1.9: Validate that the Sugar_g variable has been changed to a float.

A. Hint: Use the dtypes function.

Task #1.10: Explore the NaN values and remove the row with any NaN values.

A. Hint: Use the info() function.

Task #1.11: Assign a new dataframe df2 and remove the row with the NaN entry from the Saturated_Fat_g column in sbux1.

A. Hint: Use the dropna() function.

Task #1.12: Validate that the NaN entry from the Saturated_Fat_g column has been removed.

A. Hint: Use the info() function.

Task #1.13: Change a column name.

A. Hint: Use the rename() function.

Task #1.14: Validate that the Caffiene_mg column has been renamed corrected.

A. Hint: Use the info() function.

Task #1.15: Write dataframe sbux3 to a new sbux4.CSV file

A. Hint: Use the to_csv() function.

Task #1.16: Load the sbux.csv file back into the Jupyter Notebook and validate.

A. Hint: Use the pd.read_csv() and head() functions.

Deliverable Two: Cleaning and Preparing the Dunkin' Donuts Menu for Analysis.

Deliverable two analyzes a Dunkin' Donuts beverage menu. The objectives of deliverable two are the same as Deliverable One.

Deliverable Two Tasks:

[Click here to go back to Table of Contents](#)

Task #2.1: Import the BUS_303_Dunkin_Donuts_Nutritional_Menu_Kaggle_2023.csv file.

- A. Repeat the same steps as Task 1.1C, except import the BUS_303_Dunkin_Donuts_Nutritional_Menu_Kaggle_2023.csv file. Assign “dnkn” (in lower case letters) as the file's object name. DNKN is the stock trading ticker symbol for Dunkin’ Donuts.

Like the Starbucks CSV file, The BUS_303_Dunkin_Donuts_Nutritional_Menu_Kaggle_2023.csv file can be found in Brightspace, in the Projects/Assignments folder, or in the GitHub folder provided by your instructor. Like the Starbucks CSV file, this is a “dirty” CSV file. It contains entries that need to be cleaned. Like the Starbucks CSV file, your job is to load this nearly clean file and make further adjustments. Once the file is loaded into Python, correct the issues so it is presentable as a “clean” CSV file and ready to share with anyone.

When the run button ► is clicked, a number will appear inside the brackets [] on the left side of the code block, indicating that the file has loaded.

- B. Save the CSV file in your C:\Users\YOUR COMPUTER NAME FOLDER.

Task #2.2: Examine the Dataset Structure.

To view rows, we can use the head() function. With no entry between the (), the head function will enable one to quickly view the first five rows to assess the dataframe. However, if one wants to view a specific number of rows, specify the actual number between the ().

- A. Hint: Use the info() function.

Task #2.3: Explore Missing Values.

Missing values can negatively affect statistical analysis and visualizations. Before making any corrections, determine whether the dataset contains missing values.

Review the output and determine:

- Which columns contain missing values?
- How many missing values are present?

- A. Hint: Use the isnull().sum() function.

Task #2.4: Explore How Many Duplicate Records May Exist.

Duplicate records can distort analysis by counting the same observation multiple times.

[Click here to go back to Table of Contents](#)

How many duplicate records are present in the dataset?

A. Hint: Use the duplicated().sum() function.

Task #2.5: Explore the Nature of Duplicate Records.

You should identify exactly what duplicate records exist.

What are the duplicate records in the dataset?
Should they be removed from the dataset?

A. Hint: Use the duplicated() function.

Task #2.6: Explore Unique Values.

Business analysts often review categorical variables to identify inconsistencies, misspellings, or unusual entries.

Show the unique counts for all variables. How would you interpret this output?

A. Hint: Use the nunique() function.

Task #2.7: Examine the Beverage Categories.

Duplicate records can distort analysis by counting the same observation multiple times.

How many duplicate records are present in the dataset?

A. Hint: Use the value_counts() function.

Task #2.8: Explore Numerical Variables.

The describe() function can be used to examine continuous variables and identify unusual values.

What are the:

- Minimum values
- Maximum values
- Means
- Missing values

A. Hint: Use the describe(include='all') function.

[Click here to go back to Table of Contents](#)

Task #2.9a-i: Identify Special Characters.

Sometimes data contains special values, such as "zero", "N/A", "*", "<", "=", etc. A "special value" is any value that prevents a column from behaving as expected. For example, "zero" is not the same thing as 0. You need to identify them in the data and treat them so that a numeric column just has numbers. There are various ways to do this. You can examine each column and look for these special characters.

Identify special characters in the (a) Total Fat (g), (b) Saturated Fat (g), (c) Trans Fat (g), (d) Cholesterol (mg), (e) Sodium (mg), (f) Fiber (g), (g) Protein (g), and (i) Carbs (g).

A. Hint: Use the `.unique()` function.

Alternate method to identify special values:

Instead of identifying each column, we could use a loop to iterate over them and find special values. We are not doing it for the project. However, here is the code should you want to run it in a separate code block:

```
for col in dnkn.columns:
    if dnkn[col].dtype == 'object':
        print('\n', col)
        print(dnkn[col].unique()[:50])
```

Task #2.10: Create a Cleaned Dataframe.

Create a new dataframe named `dnkn1` that will contain all cleaning modifications. Basically, you are to copy the `dnkn` dataframe and rename the new copy.

A. Hint: Use the `copy()` function.

Task #2.11: Remove Food Items from the Category Rows.

Examine the dataset and identify rows that appear to be food category labels rather than actual beverage products.

Remove these rows from the dataframe. Do this by creating a new list of food items and then remove them from the dataset.

A. Hint: Create a new object called "food_items" and create a list inside [].

B. Use the `[~dnkn1['Category'].isin(food_items)].copy()` function.

[Click here to go back to Table of Contents](#)

Task #2.12: Verify duplicate item is removed.

Earlier, you identified a food item, Ham and Cheese Rollups, as a duplicate. When you dropped the food items, the duplicate item should have been removed. You should verify that it no longer exists in the dataframe.

A. Hint: Use the `duplicated().sum()` function.

Task #2.13a-b: Remove the special values inside each column.

Special characters were identified in the Total Fat (g), Saturated Fat (g), Trans Fat (g), Cholesterol (mg), Sodium (mg), Fiber (g), and Protein (g) columns.

You should have identified the "<5" special value in the Cholesterol (mg) column and the "\xa0" in multiple columns. The "<5" is isolated to the Cholesterol (mg) column, so that special value can be removed by just removing it from that one column.

A. Partial Hint: Use the `(dnkn2['Cholesterol (mg)'].replace('<5', '5'))` function.

For the "\xa0" special character, you could remove it one column at a time, but there is a more efficient way to do it. You can remove it from all the columns at once. This is probably the best way to do it, but it is not exactly beginner-level, so I am providing the solution for you. I highly recommend that you run this code through an AI program and prompt it to explain it to you in detail:

```
for col in dnkn2.select_dtypes(include='object').columns:
    dnkn2[col] = (
        dnkn2[col]
        .str.replace('\xa0', '', regex=False)
        .str.strip()
    )
```

Task #2.14: Validate removal of all special characters.

Normally, you would validate all columns. I am providing some code for you to validate the removal of all special values. Run the code and inspect the output.

```
for col in dnkn.columns:
    if dnkn2[col].dtype == 'object':
        print('\n', col)
        print(dnkn2[col].unique()[:50])
```

[Click here to go back to Table of Contents](#)

Task #2.15: Remove ' ' empty strings and replace with NaN values.

Now that you removed the special values from the dnkn2 dataframe, the removal of them created a " entry in your columns. This is an empty string. These must be treated. Although the special character is gone, the empty string still represents missing information. Therefore, the empty string must be converted to a NaN value so Python can properly identify and manage missing data.

The process is to convert the " to an NaN. You need to import numpy as np and replace the ' ' entries in the dnkn2 dataframe with nan values.

A. Hint: `dnkn2 = dnkn2.replace(" ", np.nan)`

Task #2.16: Validate conversion of " empty strings with NaN values.

We need to validate that the empty strings created by the removal of the '\xa0' special value have been converted to NaN values.

A. Hint: `isnull().sum()`

Task #2.17: Create a new dnkn3 object and drop the NaN values from the dataframe.

Earlier, you identified a food item, Ham and Cheese Rollups, as a duplicate. When you dropped the food items, the duplicate item should have been removed. You should verify that it no longer exists in the dataframe.

A. Hint: Use the `dropna().copy()` function.

Task #2.18: Validate removal of the NaN values from the dataframe.

Earlier, you identified a food item, Ham and Cheese Rollups, as a duplicate. When you dropped the food items, the duplicate item should have been removed. You should verify that it no longer exists in the dataframe.

A. Hint: Use the `isnull().sum()` function.

Task #2.19: Convert Data Types.

Convert the variables to numeric data types. Create a new object called "numeric_columns" and create a list of all variables that are supposed to be classified as numeric. Then, use the following code to convert them:

[Click here to go back to Table of Contents](#)

```
for col in numeric_columns:
    dnkn3[col] = pd.to_numeric(dnkn3[col])
```

Task #2.20: Summary Validations

It is a good practice to validate all items that have been cleaned up. We can summarize these with clean and neat print outs. Run the code below and if you addressed all the items properly, you will have no missing values, all numeric data types have been converted to either int64 or float64, all categories just contain beverage items, there are no special values, the data shape is (599, 13), and there are no summary statistics that are unreasonable.

Below is the code that will give you summary validations:

```
print("===== MISSING VALUES =====")
print(dnkn3.isnull().sum())

print("\n===== DATA TYPES =====")
print(dnkn3.dtypes)

print("\n===== DUPLICATE RECORDS =====")
print("Duplicates:", dnkn3.duplicated().sum())

print("\n===== CATEGORY CHECK =====")
print(dnkn3['Category'].value_counts())

print("\n===== SPECIAL VALUE CHECK =====")
print("Cholesterol Values:")
print(dnkn3['Cholesterol (mg)'].unique())

print("\nTotal Fat Values:")
print(dnkn3['Total Fat (g)'].unique())

print("\n===== DATASET SHAPE =====")
print(dnkn3.shape)

print("\n===== SUMMARY STATISTICS =====")
print(dnkn3.describe().round(2))
```

Task #2.21: Save the dnkn3 Python file to a CSV file.

A. Hint: Use the `to_csv('dnkn_clean.csv', index=False)` function.

[Click here to go back to Table of Contents](#)

Task #2.22: Reload the dnkn3 Python CSV file and save it as a new object, dnkn4.

- A. Hint: Use the `pd.read_csv('dnkn_clean.csv')` function.
- B. Hint: Validate with the `head()` function.

Task #2.22: Summarize the issues with the Dunkin' Donuts dataset.

In a two or three-paragraph layman's summary, tell your client what you did for Deliverable Two.

Enter it in your Jupyter Notebook.

Deliverable Three: Comparing Starbucks and Dunkin' Donuts Menu for a Competitive Analysis

Deliverable Three Tasks:

Task #3.1: Create a horizontal bar chart comparing the Beverage Combination Portfolio Size of Starbucks and Dunkin' Donuts.

Create a horizontal bar chart comparing the Beverage Combination Portfolio Size of Starbucks and Dunkin' Donuts.

Try to include the following:

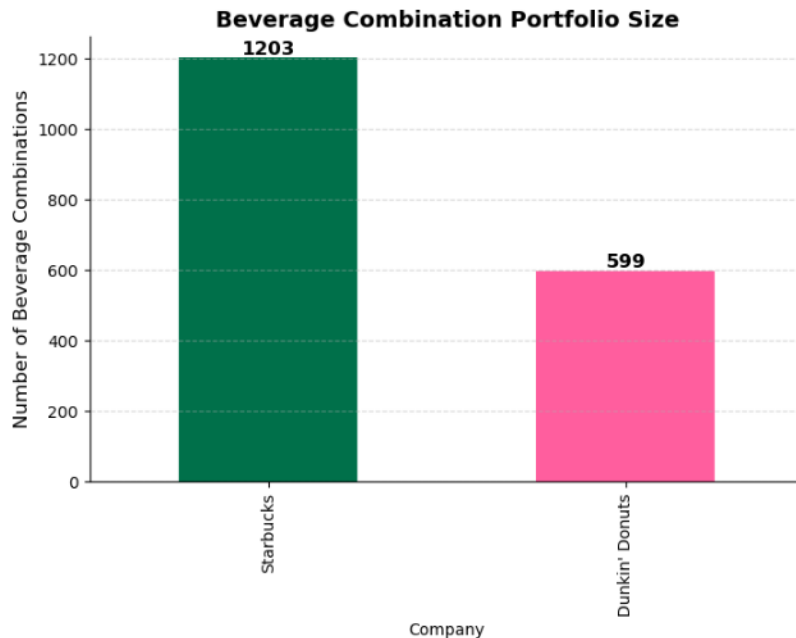
- Data labels showing the number of beverages on each bar.
- Include title, x-label, and y-label.
- Use the official exact colors of Starbucks (green) and Dunkin' Donuts (pink) for the chart.

One of the simplest ways companies compete is through product variety. This visualization compares the number of beverage combination offerings available at Starbucks and Dunkin' Donuts.

- What are some of the pros and cons of a variety of beverage combinations?

A. Hint: It should look like this:

[Click here to go back to Table of Contents](#)



B. Hint: Here is some partial code to assist you:

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
portfolio = pd.DataFrame({
    "Company": ["Starbucks", "Dunkin' Donuts"],
    "Beverage_combinations": [len(sbox4), len(dnkn4)] # actual counts from your data
})
```

```
# Define the official colors for Starbucks and Dunkin' Donuts
starbucks_green = "#00704A"
dunkin_pink = "#FF5E9E" # Dunkin' bright pink
colors = [starbucks_green, dunkin_pink]
```

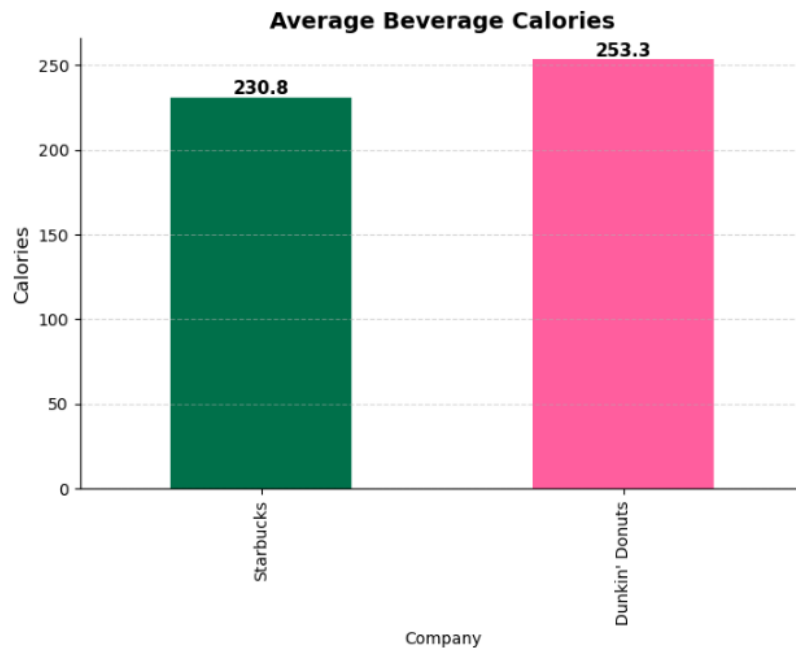
Task #3.2: Create a bar chart comparing average beverage calories.

Try to include the following:

- Average Calories per Beverage
- Calculate the average calories for all beverages of each company.
- Create a bar chart to compare Starbucks vs Dunkin' Donuts.
- Use brand colors consistently.
- Add value labels for the average calories on each bar.

[Click here to go back to Table of Contents](#)

A. Hint: It should look like this:



B. Hint: Here is some partial code to assist you:

```
import pandas as pd
import matplotlib.pyplot as plt

# Calculate average calories for each menu.
avg_calories = pd.DataFrame({
    "Company": ["Starbucks", "Dunkin' Donuts"],
    "Average_Calories": [
        sbux4["Calories"].mean(),
        dnkn4["Calories"].mean()
    ]
})

# Define the official colors for Starbucks and Dunkin' Donuts
starbucks_green = "#00704A"
dunkin_pink = "#FF5E9E"

# Create a Horizontal Bar chart
ax = avg_calories.plot(
    kind="bar",
    x="Company",
    y="Average_Calories",
    color=[starbucks_green, dunkin_pink],
```

[Click here to go back to Table of Contents](#)

```

figsize=(8,5),
legend=False
)

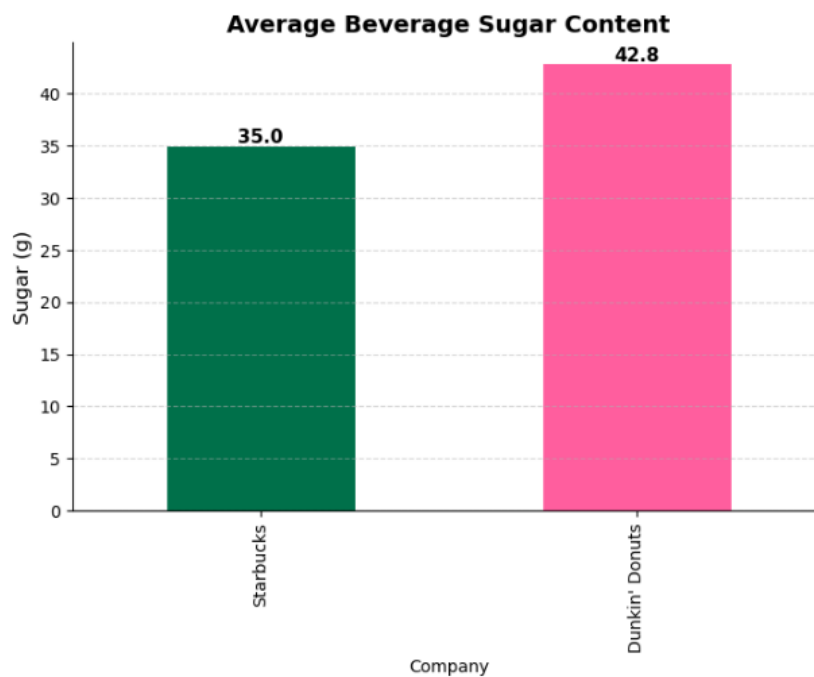
```

Task #3.3: Create a bar chart comparing average sugar content.

Compute the Average Sugar (grams) Content for beverages of each company.

- Visualize with a bar chart comparing Starbucks and Dunkin' Donuts.
- Include data labels, title, and axis labels.

A. Hint: It should look something like this:



B. Hint: Here is some partial code to assist you:

```

# Calculate the average sugar for each menu.
avg_sugar = pd.DataFrame({
    "Company": ["Starbucks", "Dunkin' Donuts"],
    "Average_Sugar": [
        sbux4["Sugar_g"].mean(),
        dnkn4["Sugars (g)"].mean()
    ]
})

```

Create a Horizontal Bar chart

[Click here to go back to Table of Contents](#)


```

ax = avg_sugar.plot(
    kind="bar",
    x="Company",
    y="Average_Sugar",
    color=[starbucks_green, dunkin_pink],
    figsize=(8,5),
    legend=False
)

# Add Titles & labels
plt.title("Average Beverage Sugar Content", fontsize=14, fontweight="bold")
plt.ylabel("Sugar (g)", fontsize=12)

for p in ax.patches:
    ax.annotate(
        f'{p.get_height():.1f}',
        (p.get_x()+p.get_width()/2., p.get_height()),
        ha="center",
        va="bottom",
        fontsize=11,
        fontweight="bold"
    )

```

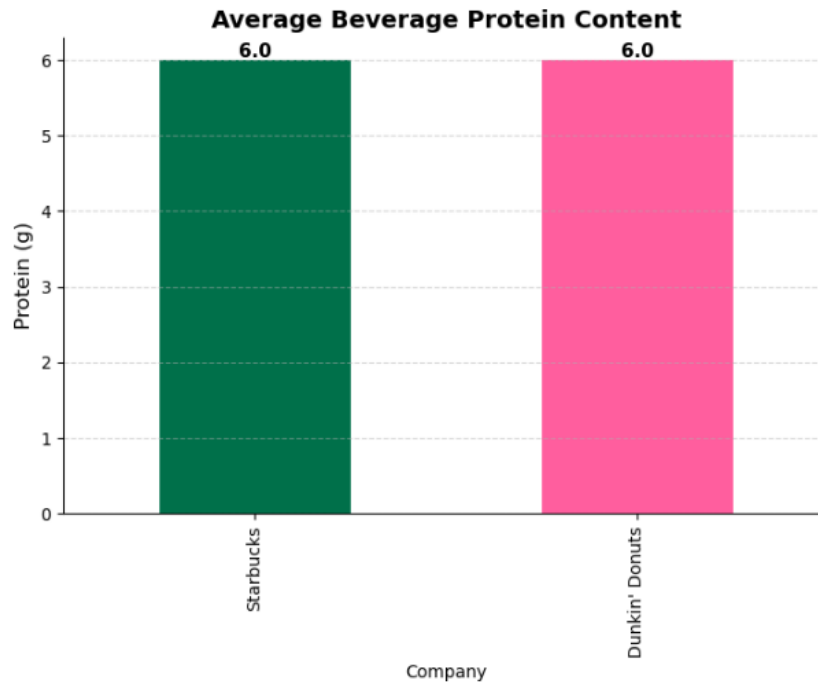
Task #3.4: Create a bar chart comparing average protein content.

Calculate the average protein content for beverages of each company.

- Display a bar chart comparison with official brand colors. Include value labels and proper axis labels.

A. Hint: It should look something like this:

[Click here to go back to Table of Contents](#)



B. Hint: Here is some partial code to assist you:

```
# Calculate average protein for each menu.
protein = pd.DataFrame({
    "Company": ["Starbucks", "Dunkin' Donuts"],
    "Average_Protein": [
        sbux4["Protein_g"].mean(),
        dnkn4["Protein (g)"].mean()
    ]
})
```

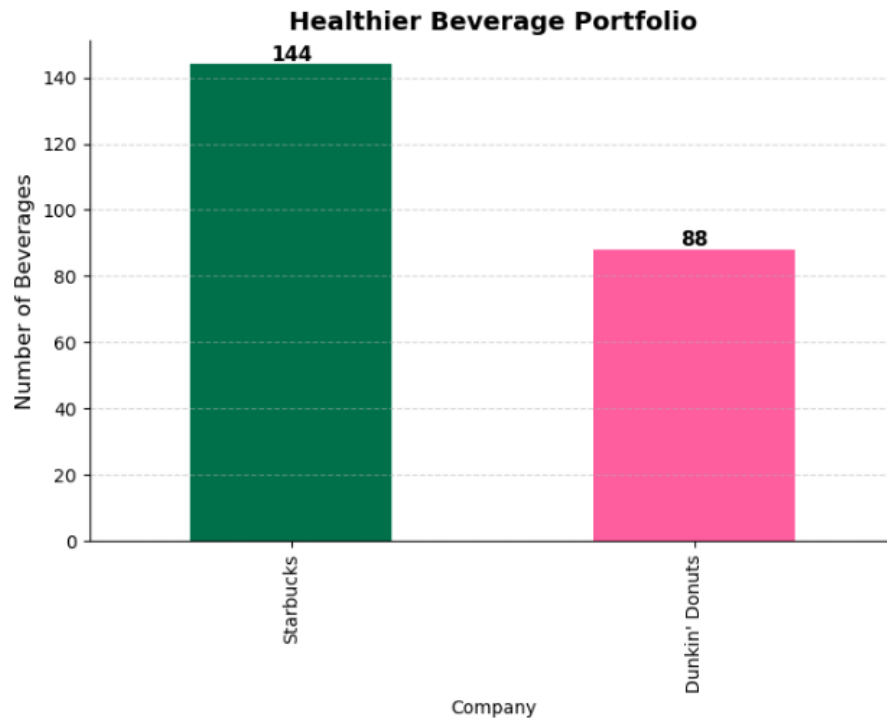
```
# Create a Horizontal Bar chart
ax = protein.plot(
    kind="bar",
    x="Company",
    y="Average_Protein",
    color=[starbucks_green, dunkin_pink],
    figsize=(8,5),
    legend=False
)
```

Task #3.5: Compare healthier beverage choices.

[Click here to go back to Table of Contents](#)

Define healthy beverages as Calories < 100 and Sugar < 15g. Count how many healthy beverages each company offers. Create a simple bar chart to display the counts. Use Starbucks green and Dunkin orange/pink. Add value labels for each bar.

A. Hint: It should look something like this:



B. Hint: Here is some partial code to assist you:

Define a threshold for what is acceptably healthy in terms of calories and sugar for Starbucks.

```
sbux_healthy = len(
    sbux4[
        (sbux4["Calories"] < 100) &
        (sbux4["Sugar_g"] < 15)
    ]
)
```

Define a threshold for what is acceptably healthy in terms of calories and sugar for Dunkin' Donuts.

```
dnkn_healthy = len(
    dnkn4[
        (dnkn4["Calories"] < 100) &
        (dnkn4["Sugars (g)"] < 15)
    ]
)
```

[Click here to go back to Table of Contents](#)

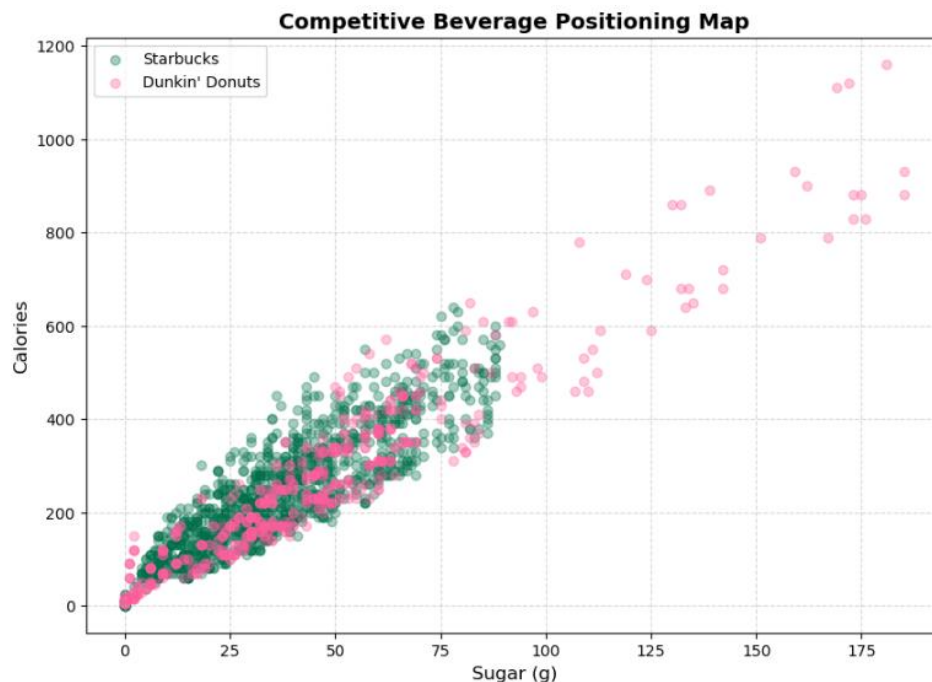
)

```
# Create a dataframe of the number of healthy drink combinations for both menus.
healthy = pd.DataFrame({
    "Company":["Starbucks","Dunkin' Donuts"],
    "Healthy_Beverages":[
        sbux_healthy,
        dnkn_healthy
    ]
})
```

Task #3.6: Competitive Positioning Map.

Create a scatter plot with Calories on the y-axis and Sugar on the x-axis. Plot Starbucks beverages and Dunkin' beverages in separate colors. Include a title, axis labels, and legend to distinguish the companies. Use transparency (alpha) to visualize overlapping points.

A. Hint: It should look something like this:



B. Hint: Here is some partial code to assist you:

```
# Create a figure and define the overall size of the chart.
plt.figure(figsize=(10,7))
```

```
# Plot Starbucks beverages using sugar on the x-axis and calories on the y-axis.
```

[Click here to go back to Table of Contents](#)

```

# Each point represents a beverage combination.
plt.scatter(
    sbux4["Sugar_g"],
    sbux4["Calories"],
    color=starbucks_green,
    alpha=.35,
    label="Starbucks"
)

# Plot Dunkin' Donuts beverages using sugar on the x-axis and calories on the y-axis.
# Each point represents a beverage.
plt.scatter(
    dnkn4["Sugars (g)"],
    dnkn4["Calories"],
    color=dunkin_pink,
    alpha=.35,
    label="Dunkin' Donuts"
)

```

Task #3.7: Zero-Calorie Beverages Count Analysis.

Count how many zero-calorie beverages each company offers.

A. Hint: Here is some partial code to assist you:

```

# Starbucks zero-calorie beverages
sbux_zero_cal = sbux4[sbux4["Calories"] == 0]

# Dunkin zero-calorie beverages
dnkn_zero_cal = dnkn4[dnkn4["Calories"] == 0]

```

Task #3.8: Zero-Calorie Starbucks Beverage Analysis.

Filter all Starbucks beverages with 0 calories. Visualize with a table comparing showing each product name, size, milk content, whip cream or not, calories, sugar and protein content.

A. Here is some partial code to assist you:

```

sbux_zero_cal[
    [
        "Product_Name",
    ]
]

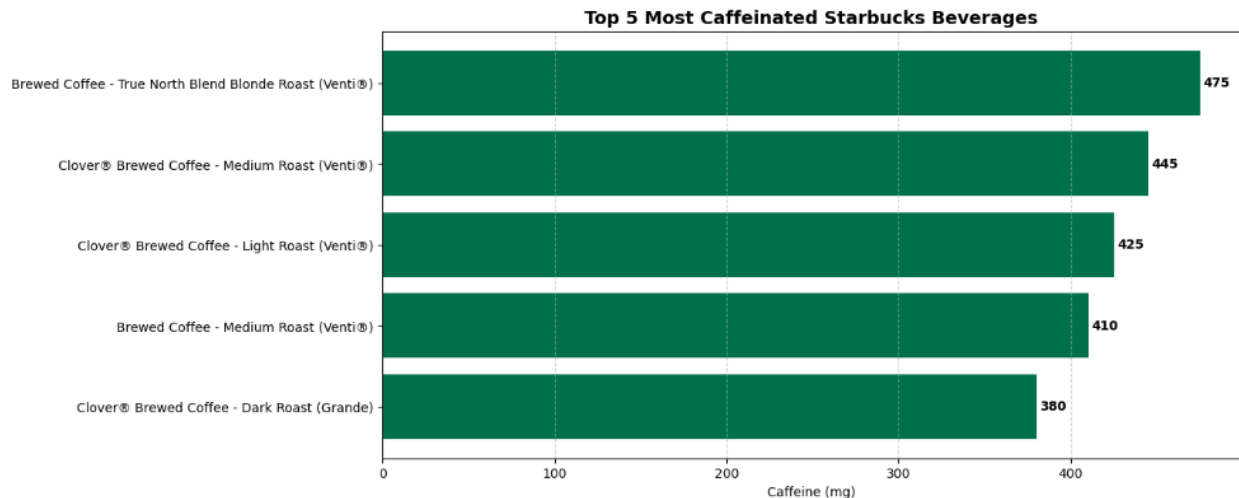
```

[Click here to go back to Table of Contents](#)

Task #3.9: Top 5 Most Caffeinated Starbucks Beverages.

Identify the top 5 beverages with the highest caffeine content for Starbucks and Dunkin'. Include the size of each beverage in the label. Display horizontal bar charts with brand colors. Add caffeine values as data labels.

A. Hint: It should look something like this:



B. Hint: Here is some partial code to assist you:

```
# Import libraries.
import matplotlib.pyplot as plt

# Sort by caffeine and take top 5
top_caffeine_sbux = sbux4.sort_values(
    by='Caffeine_mg', ascending=False
).head(5)

# Combine Product Name and Size for labels
top_caffeine_sbux['Label'] = top_caffeine_sbux['Product_Name'] + " (" +
top_caffeine_sbux['Size'] + ")"

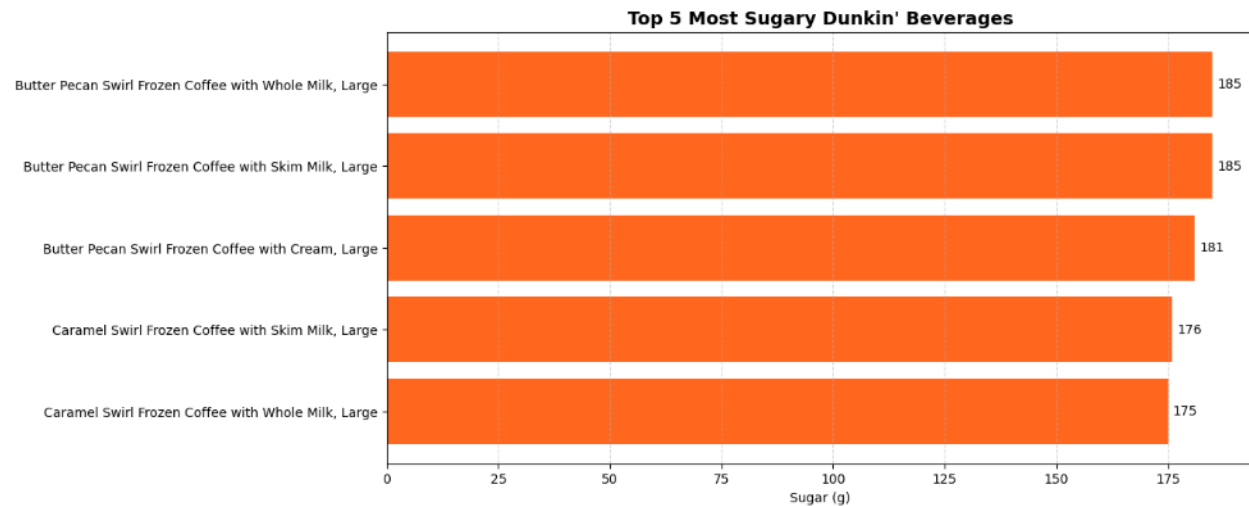
# Horizontal bar chart
plt.figure(figsize=(12,6))
plt.barh(
    top_caffeine_sbux['Label'],
    top_caffeine_sbux['Caffeine_mg'],
    color="#00704A" # Starbucks green
)
```

[Click here to go back to Table of Contents](#)

Task #3.10: Top 5 Most Sugary Dunkin' Donuts Beverages.

- Identify the top 5 beverages with the highest sugar content for Dunkin' Donuts.
- Include the size of each beverage in the label.
- Display horizontal bar charts with brand colors.
- Add sugar (g) values as data labels.

A. Hint: It should look something like this:



B. Hint: Here is some partial code to assist you:

```
# Sort beverages by sugar content and select the top 5
top_sugary_dnkn = dnkn4.sort_values(
    by='Sugars (g)',
    ascending=False
).head(5)
```

```
# Create horizontal bar chart
plt.figure(figsize=(12,6))
```

```
plt.barh(
    top_sugary_dnkn['Item'],
    top_sugary_dnkn['Sugars (g)'],
    color="#FF671F"
)
```

```
# Plot sugar content for each beverage
plt.title(
    "Top 5 Most Sugary Dunkin' Beverages",
    fontsize=14,
```

[Click here to go back to Table of Contents](#)


```
fontweight='bold'  
)
```

Task #3.11: Choose your own visual.

Choose your own visual.

Create a Markdown block explaining what you are trying to accomplish and then underneath it, show and execute your Python code.

There are many observations to note about the variables that your client may be interested in hearing about. Here are just a few ideas regarding Starbucks' inventory (however, you have to come up with your own ideas - not these!).

- The minimum size for any Starbucks drink is 236 ml, which is about 8 ounces. The maximum is 887 ml, which is about 30 ounces. Could there be opportunities to serve smaller or larger drinks? Why did Starbucks choose these drink size ranges?
- Every nutritional measure has a minimum zero count. Show it. Starbucks drinks tend to have more cholesterol than fewer. Look at the 0% to 50% range compared to the 75% to 100% range.
- White Hot Chocolate is the most frequently repeated Product Name. Show it.
- There are six different types of Milk options. Show which ones have the least fat.

Deliverable Four: Short Video Explaining Deliverables One through Three.

The objective of Deliverable Four is to enable one to learn how to present business data analyzed through Python to make decisions.

- You are to present a walkthrough of your Jupyter Notebook and comment on the key findings/issues for your work product. Keep in mind that you will be submitting your Jupyter Notebook, and the instructor can access it via your GitHub account. You do not need to explain every detail, only key findings/issues.
- Follow the instructions in the Step-by-Step guide to submit it.
- Upload the video presentation or link it to Brightspace.

Task #4.1: Rehearse, record, and present a video presentation.

An important part of business analytics is not only to present information in an easy to understand format, but to present how information is processed. It is important to rehearse the presentation a few times before recording. The recording should be as concise as possible to keep the file size manageable. Try to keep it under five minutes. You can present it in any medium as long as it can be opened by the instructor. Some suggestions include using a narrated

[Click here to go back to Table of Contents](#)

PowerPoint, Zoom, Canva, SnagIt, Camtasia, etc. You can upload it to YouTube, Panopto, or another location where you can easily share and play video files.

To assist students in developing a narrated PowerPoint, consider the following video examples:

- How to Record Narration for a PowerPoint Presentation For Dummies:
<https://www.youtube.com/watch?v=QZp3jumnWUg>

Task #4.2: Upload the video presentation.

Upload the video or provide a link to the video in Brightspace.

Deliverable Five: Reflection and Final Submission.

The objective of deliverable five is to enable one to reflect with a critical review of what you learned in the course and how Python can build your skill set.

Task #5.1: Write a reflection in the Python file.

In the Deliverable Five markdown block, write a reflection, then copy and paste it into a separate Word document.

Students write a reflection with a critical review of what you learned in the course and how Python can build a desired skill set. Be mindful to. I am very interested to hear the following:

- Review the course learning objectives in the syllabus and discuss how you feel the course and the Signature Term Project achieved those objectives.
- What you expected before you came into this class and what you think you are taking away from this class.
- Your perception about Python and programming and how it may be able to help you better understand abstract business information and develop your skill set. What was easy and challenging as far as learning actual programming? Why Is Python interesting to you? Did Datacamp help you?
- What would you have like to have learned that was not covered?
- How could the directions be clearer for the Signature Term Project?
- Do you think that the hints made the project less challenging? Without them, do you think that you could have completed the project by the conclusion of the semester?
- Did you think that understanding GitHub was important? Although we did not even come close to understanding its capabilities, from your exposure do you think it will become one of your tools to communicate workflows? How will you use it?
- Do you plan to study Python further? How?

[Click here to go back to Table of Contents](#)

I am looking forward to your reflections and I read every word that you write!

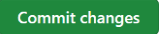
Task #5.2: Print an HTML copy of the Jupyter Notebook.

Go to File/Save and Export Notebook As/HTML. It will print to your download folder. Upload it to your GitHub account so your instructor can access it.

Task #5.3: Convert the HTML to a PDF file to upload it to ePortfolio.

Open up the HTML file and follow your browser's instructions to convert it to a PDF.

Task #5.4: Email the GitHub links to yourself, your instructor and upload it into Blackboard.

Sign into your GitHub account and go into your repository that contains your newly created Jupyter Notebook Source File (.ipynb) and your HTML files. Click on the  icon and choose upload files. Once you choose your file from the C:\Users\YOUR COMPUTER NAME FOLDER (For MAC users, from the /Users/ YOUR COMPUTER NAME FOLDER), once uploaded, make sure that you scroll down the page and hit the  icon to complete the upload. The file should now be in your repository to share with the world. To share the file, click on the file name in the left panel and copy the URL.

Task #5.5: Convert the HTML to a PDF file to upload it to ePortfolio.

For this task:

1. Email the GitHub link to yourself.
2. Email the GitHub link to your instructor.
3. Upload it into Blackboard so it can be graded.

[Click here to go back to Table of Contents](#)

Appendix A, Grading Rubrics

Deliverable One Grading Rubric				
Levels of Achievement				
Task	Accomplished	Proficient	Developing	Beginning
1.1 Import Starbucks dataset	Successfully imports dataset, notebook is properly named, code runs without errors, and dataframe loads correctly.	Dataset imports with minor naming, formatting, or coding issues.	Dataset imports but significant errors or incomplete execution are present.	Missing or minimal evidence.
1.2 Explore blank values	Correctly identifies all blank values and accurately interprets findings.	Identifies most blank values with minor interpretation issues.	Partial identification of blank values or inaccurate interpretation.	Missing or minimal evidence.
1.3 Examine dataset with head()	Uses head() correctly and demonstrates understanding of dataset structure and contents.	Uses head() correctly with limited interpretation.	Uses head() incorrectly or demonstrates limited understanding.	Missing or minimal evidence.
1.4 Remove blank rows	Correctly removes all blank rows while preserving valid records.	Removes blank rows with minor coding or procedural issues.	Attempts removal but blank rows remain or valid records are affected.	Missing or minimal evidence.
1.5 Validate blank-row removal	Validation clearly demonstrates successful removal of blank rows.	Validation performed with minor omissions.	Validation incomplete or unclear.	Missing or minimal evidence.
1.6 Explore data types	Correctly identifies data types and	Identifies most data types with limited discussion.	Reviews data types but misses important issues.	Missing or minimal evidence.

[Click here to go back to Table of Contents](#)

	discusses data-quality concerns.			
1.7 Identify incorrect data type	Correctly identifies incorrect data type and explains why correction is needed.	Correctly identifies variable with limited explanation.	Incorrect variable identified or explanation is weak.	Missing or minimal evidence.
1.8 Convert Sugar_g to float	Correctly converts Sugar_g to float using appropriate syntax.	Conversion completed with minor coding issues.	Conversion attempted but contains errors.	Missing or minimal evidence.
1.9 Validate Sugar_g conversion	Validation clearly confirms successful conversion.	Validation confirms conversion with minor omissions.	Validation incomplete or unclear.	Missing or minimal evidence.
1.10 Explore NaN values	Correctly identifies missing values and explains analytical implications.	Identifies missing values with minor interpretation issues.	Partial identification of missing values.	Missing or minimal evidence.
1.11 Remove NaN row	Correctly removes the NaN row while preserving valid records.	Removes NaN row with minor procedural issues.	Attempts removal but errors remain.	Missing or minimal evidence.
1.12 Validate NaN removal	Validation clearly demonstrates successful NaN removal.	Validation completed with minor omissions.	Validation incomplete or inaccurate.	Missing or minimal evidence.
1.13 Rename column	Correctly renames specified column using appropriate conventions.	Column renamed with minor coding or formatting issues.	Renaming attempted but contains errors.	Missing or minimal evidence.
1.14 Validate renamed column	Validation clearly confirms successful renaming.	Validation confirms rename with minor omissions.	Validation incomplete or unclear.	Missing or minimal evidence.

[Click here to go back to Table of Contents](#)

1.15 Save sbux3 to CSV	Correctly exports sbux3 to CSV using appropriate settings.	CSV exported with minor formatting issues.	Export attempted but file contains errors.	Missing or minimal evidence.
1.16 Reload CSV and validate	Successfully reloads CSV, verifies integrity, and confirms it matches cleaned dataset.	CSV reloaded with minor validation omissions.	Reload attempted but validation is incomplete.	Missing or minimal evidence.

Deliverable Two Grading Rubric				
Levels of Achievement				
Task	Accomplished	Proficient	Developing	Beginning
2.1 Import the Dunkin dataset	Successfully imports dataset, code runs without errors, and dataframe loads correctly.	Dataset imports with minor coding or formatting issues.	Dataset imports but significant errors or incomplete execution are present.	Missing or minimal evidence.
2.2 Examine dataset structure	Accurately interprets rows, columns, variables, and overall structure.	Examines structure with minor interpretation errors.	Partial examination or inaccurate interpretation.	Missing or minimal evidence.
2.3 Explore missing values	Correctly identifies all missing values and explains implications.	Identifies most missing values with minor omissions.	Partial identification of missing values.	Missing or minimal evidence.
2.4 Explore duplicate records	Correctly identifies and quantifies duplicate records.	Identifies duplicates with minor reporting errors.	Partial duplicate analysis.	Missing or minimal evidence.
2.5 Explore nature of duplicate records	Evaluates duplicates and provides logical rationale for retention/removal.	Evaluation is generally correct with minor omissions.	Limited evaluation or weak rationale.	Missing or minimal evidence.

[Click here to go back to Table of Contents](#)

2.6 Explore unique values	Correctly identifies and interprets unique values across variables.	Identifies unique values with minor omissions.	Partial exploration of unique values.	Missing or minimal evidence.
2.7 Examine beverage categories	Accurately identifies beverage categories and interprets findings.	Categories identified with minor interpretation issues.	Incomplete category analysis.	Missing or minimal evidence.
2.8 Explore numerical variables	Evaluates numerical variables and identifies unusual values.	Evaluates variables with minor omissions.	Limited numerical analysis.	Missing or minimal evidence.
2.9a Identify special characters in Total Fat	Correctly identifies all special characters, symbols, and data-quality issues in Total Fat.	Identifies most special characters with only minor omissions.	Partial identification of special characters or data issues.	Missing or minimal evidence.
2.9b Identify special characters in Saturated Fat	Correctly identifies all special characters, symbols, and data-quality issues in Saturated Fat.	Identifies most special characters with only minor omissions.	Partial identification of special characters or data issues.	Missing or minimal evidence.
2.9c Identify special characters in Trans Fat	Correctly identifies all special characters, symbols, and data-quality issues in Trans Fat.	Identifies most special characters with only minor omissions.	Partial identification of special characters or data issues.	Missing or minimal evidence.
2.9d Identify special characters in Cholesterol	Correctly identifies all special characters, symbols, and data-quality issues in Cholesterol.	Identifies most special characters with only minor omissions.	Partial identification of special characters or data issues.	Missing or minimal evidence.

[Click here to go back to Table of Contents](#)

2.9e Identify special characters in Sodium	Correctly identifies all special characters, symbols, and data-quality issues in Sodium.	Identifies most special characters with only minor omissions.	Partial identification of special characters or data issues.	Missing or minimal evidence.
2.9f Identify special characters in Fiber	Correctly identifies all special characters, symbols, and data-quality issues in Fiber.	Identifies most special characters with only minor omissions.	Partial identification of special characters or data issues.	Missing or minimal evidence.
2.9g Identify special characters in Protein	Correctly identifies all special characters, symbols, and data-quality issues in Protein.	Identifies most special characters with only minor omissions.	Partial identification of special characters or data issues.	Missing or minimal evidence.
2.9h Identify special characters in Carbs	Correctly identifies all special characters, symbols, and data-quality issues in Carbs.	Identifies most special characters with only minor omissions.	Partial identification of special characters or data issues.	Missing or minimal evidence.
2.10 Create a cleaned dataframe	Creates cleaned dataframe correctly while preserving valid records.	Creates dataframe with minor procedural issues.	Dataframe contains significant cleaning errors.	Missing or minimal evidence.
2.11 Remove food items	Correctly removes all non-beverage records.	Removes most food items with minor errors.	Partial removal of food items.	Missing or minimal evidence.
2.12 Verify duplicate item removal	Validation clearly confirms duplicate removal.	Validation performed with minor omissions.	Validation incomplete or unclear.	Missing or minimal evidence.
2.13 Remove special values	Correctly removes or resolves all special values.	Removes most special values with minor issues.	Partial removal of special values.	Missing or minimal evidence.

[Click here to go back to Table of Contents](#)

2.14 Validate special value removal	Validation clearly demonstrates successful removal.	Validation completed with minor omissions.	Validation incomplete or inaccurate.	Missing or minimal evidence.
2.15 Convert empty strings to NaN values	Correctly converts empty strings to NaN values using appropriate methods.	Conversion completed with minor issues.	Conversion attempted but incomplete.	Missing or minimal evidence.
2.16 Validate conversion to NaN values	Validation clearly confirms successful conversion.	Validation performed with minor omissions.	Validation incomplete or unclear.	Missing or minimal evidence.
2.17 Create dnkn3 and remove NaN values	Correctly creates dnkn3 and removes all required NaN values.	Completed with minor procedural issues.	Attempted but errors remain.	Missing or minimal evidence.
2.18 Validate NaN removal	Validation clearly demonstrates successful NaN removal.	Validation completed with minor omissions.	Validation incomplete or inaccurate.	Missing or minimal evidence.
2.19 Convert data types	Correctly converts all required variables to appropriate data types.	Most conversions completed correctly.	Partial conversion or significant errors remain.	Missing or minimal evidence.
2.20 Perform summary validations	Performs comprehensive validations confirming dataset quality.	Validation performed with minor omissions.	Validation incomplete or partially correct.	Missing or minimal evidence.
2.21 Save dnkn3 to CSV	Correctly exports dnkn3 to CSV using appropriate settings.	CSV exported with minor formatting issues.	Export attempted but file contains errors.	Missing or minimal evidence.

[Click here to go back to Table of Contents](#)

2.22 Reload dnkn3 CSV as dnkn4	Successfully reloads CSV and verifies data integrity.	CSV reloaded with minor validation omissions.	Reload attempted but validation incomplete.	Missing or minimal evidence.
2.23 Summarize issues with the Dunkin dataset	Provides thorough summary of data issues, cleaning decisions, and analytical implications.	Summary addresses major issues with minor omissions.	Summary incomplete or largely descriptive.	Missing or minimal evidence.

Deliverable Three Grading Rubric				
Levels of Achievement				
Task	Accomplished	Proficient	Developing	Beginning
3.1 Beverage Portfolio Size Analysis	Creates an accurate visualization comparing beverage portfolio size and provides meaningful competitive interpretation.	Creates an accurate visualization with adequate interpretation.	Visualization present but contains errors or limited interpretation.	Missing or minimal evidence of attempting the task.
3.2 Average Beverage Calories Analysis	Accurately analyzes average calories and explains competitive implications using appropriate visualization.	Accurately analyzes calories with limited interpretation.	Analysis contains errors or interpretation is incomplete.	Missing or minimal evidence of attempting the task.
3.3 Average Sugar Content Analysis	Accurately analyzes average sugar content and provides meaningful business interpretation.	Analysis is accurate with minor interpretation omissions.	Analysis contains errors or limited interpretation.	Missing or minimal evidence of attempting the task.
3.4 Average Protein Content Analysis	Accurately analyzes average protein content and explains	Analysis is accurate with minor	Analysis contains errors or interpretation is incomplete.	Missing or minimal evidence of attempting the task.

[Click here to go back to Table of Contents](#)

	competitive implications.	interpretation omissions.		
3.5 Healthier Beverage Choices Analysis	Correctly identifies healthier beverage options and supports conclusions using data.	Identifies healthier options with minor interpretation issues.	Conclusions are only partially supported by data.	Missing or minimal evidence of attempting the task.
3.6 Competitive Positioning Map	Creates an accurate positioning map and clearly explains competitive positioning.	Positioning map is accurate with minor interpretation issues.	Map contains errors or limited interpretation.	Missing or minimal evidence of attempting the task.
3.7 Zero-Calorie Beverage Count Analysis	Accurately analyzes zero-calorie beverage counts and explains competitive significance.	Analysis is accurate with limited discussion.	Analysis contains errors or incomplete interpretation.	Missing or minimal evidence of attempting the task.
3.8 Zero-Starbucks Calorie Beverage Analysis	Correctly analyzes Starbucks zero-calorie offerings and provides meaningful insights.	Analysis is accurate with minor omissions.	Analysis contains errors or limited interpretation.	Missing or minimal evidence of attempting the task.
3.9 Top 5 Most Caffeinated Starbucks Beverages	Correctly identifies the top five beverages, creates an effective visualization, and interprets findings.	Correctly identifies beverages with minor visualization or interpretation issues.	Analysis contains errors or incomplete interpretation.	Missing or minimal evidence of attempting the task.
3.10 Top 5 Most Sugary Dunkin Donuts Beverages	Correctly identifies the top five beverages, creates an effective visualization, and interprets findings.	Correctly identifies beverages with minor visualization or interpretation issues.	Analysis contains errors or incomplete interpretation.	Missing or minimal evidence of attempting the task.

[Click here to go back to Table of Contents](#)

3.11 Student-Created Visualization	Creates an original visualization that provides meaningful business insight beyond required analyses.	Creates an original visualization with adequate explanation.	Visualization provides limited insight or contains errors.	Missing or minimal evidence of attempting the task.
------------------------------------	---	--	--	---

Deliverable Four Grading Rubric				
Levels of Achievement				
Task	Accomplished	Proficient	Developing	Beginning
4.1 Rehearse, Record, and Present Video	Delivers a professional, well-organized presentation that clearly explains the data cleaning process, competitive analysis findings, visualizations, and business insights. Speaker demonstrates mastery of the project and communicates effectively.	Delivers a clear presentation that explains most aspects of the project accurately with only minor omissions or communication issues.	Presentation addresses some project components but contains significant omissions, organizational issues, or inaccurate explanations.	Missing presentation or minimal evidence of preparation and understanding.
4.2 Upload Video Presentation	Video is uploaded successfully, follows all submission requirements, functions correctly, and is accessible to the instructor.	Video is uploaded successfully with only minor formatting, naming, or accessibility issues.	Video submission contains significant technical, formatting, or accessibility issues.	Video not submitted or inaccessible to the instructor.

[Click here to go back to Table of Contents](#)

Deliverable Five Grading Rubric				
Levels of Achievement				
Task	Accomplished	Proficient	Developing	Beginning
5.1 Write Reflection	Provides a thoughtful and comprehensive reflection that discusses learning, challenges, problem-solving strategies, business insights, and the application of Python and data analytics skills.	Provides a clear reflection addressing most major learning outcomes with only minor omissions.	Reflection is largely descriptive, incomplete, or demonstrates limited self-assessment.	Reflection missing or demonstrates minimal effort and insight.
5.2 Print HTML Copy of Jupyter Notebook	Successfully generates an HTML version of the notebook that is complete, properly formatted, and accurately reflects the final project.	HTML file generated successfully with only minor formatting or completeness issues.	HTML file generated but contains significant formatting, content, or technical issues.	HTML file not generated or submitted.
5.3 Convert HTML to PDF	Successfully converts the HTML file to PDF, preserving formatting, readability, and project content.	PDF generated successfully with only minor formatting issues.	PDF generated but contains significant formatting, readability, or content issues.	PDF not generated or submitted.
5.4 Email GitHub Links and Upload to Brightspace	Correctly emails all required GitHub links, follows submission	Submission completed with only minor omissions or	Submission incomplete or contains significant errors	Required materials not submitted.

[Click here to go back to Table of Contents](#)

	instructions, and uploads all required materials successfully.	formatting issues.	in links, files, or instructions.	
5.5 Convert HTML to PDF for ePortfolio Submission	Successfully prepares and submits a professional-quality PDF suitable for ePortfolio use that accurately represents the completed project.	PDF prepared successfully with only minor formatting or presentation issues.	PDF prepared but contains significant formatting, completeness, or presentation issues.	ePortfolio PDF not submitted or incomplete.

###

[Click here to go back to Table of Contents](#)